@tetherto/wdk-protocol-fiat-transak

Note: This package is in beta. Please test in a dev setup first.

Integrate the Transak widget for buying (on-ramp) and selling (off-ramp) crypto. Use it to generate widget URLs, get buy and sell quotes, and look up supported currencies, countries, and order details. Works in both frontend and backend code.

🔍 About WDK

This is part of WDK (Wallet Development Kit). WDK helps you build safe, non-custody wallets. Read more at https://docs.wallet.tether.io.

🌟 Features

- Generate a widget URL to buy crypto (on-ramp)
- Generate a widget URL to sell crypto (off-ramp)
- Get buy and sell quotes
- List supported currencies and countries
- Look up order (transaction) details

⬇ Installation

```
npm install @tetherto/wdk-protocol-fiat-transak
```

🚀 Quick Start

Basic Usage

```
import TransakProtocol from '@tetherto/wdk-protocol-fiat-transak'

// `widgetUrl` receives the assembled widgetParams and returns a widget URL.
// Do the session creation on your backend, where your API secret lives.
const widgetUrl = async (widgetParams) => {
  const response = await fetch('https://your-backend.example.com/transak/widget-url', {
    method: 'POST',
    headers: {
      'Content-Type': 'application/json'
    },
```

```
    body: JSON.stringify({ widgetParams })
  })

  if (!response.ok) {
    throw new Error(`Failed to create Transak widget URL: ${response.status} ${response.statu
  }

  const { widgetUrl } = await response.json()

  return widgetUrl
}

// Initialize protocol without a wallet account.
// You then pass `recipient`/`refundAddress` explicitly on buy/sell (see below).
const transak = new TransakProtocol(undefined, {
  apiKey: 'YOUR_TRANSAK_PARTNER_KEY',
  widgetUrl,
  environment: 'STAGING'
})

// Amounts are in base units: fiat in cents, crypto in wei.

// Get a buy quote
const buyQuote = await transak.quoteBuy({
  cryptoAsset: 'ETH',
  fiatCurrency: 'USD',
  fiatAmount: 10_000n // 100 USD (in cents)
})

// Generate a buy widget URL
const { buyUrl } = await transak.buy({
  cryptoAsset: 'ETH',
  fiatCurrency: 'USD',
  fiatAmount: 10_000n, // 100 USD (in cents)
  recipient: '0xabc'
})

console.log('Buy URL:', buyUrl)

// Get a sell quote
const sellQuote = await transak.quoteSell({
  cryptoAsset: 'ETH',
  fiatCurrency: 'USD',
  cryptoAmount: 100_000_000_000_000_000n // 0.1 ETH (in wei)
})

// Generate a sell widget URL
const { sellUrl } = await transak.sell({
  cryptoAsset: 'ETH',
  fiatCurrency: 'USD',
  cryptoAmount: 100_000_000_000_000_000n, // 0.1 ETH (in wei)
  refundAddress: '0xabc'
})
```

Using a Wallet Account

The first constructor argument is an optional WDK wallet account. When you bind one, `buy` and `sell` automatically use the account's address, so you don't need to pass `recipient`/`refundAddress` on every call:

```
import TransakProtocol from '@tetherto/wdk-protocol-fiat-transak'

// `walletManager` is your app's WDK wallet manager (chain-specific).
// getAccount() returns an IWalletAccount exposing getAddress().
const account = await walletManager.getAccount(0)

const transak = new TransakProtocol(account, {
  apiKey: 'YOUR_TRANSAK_PARTNER_KEY',
  widgetUrl,
  environment: 'STAGING'
})

// No `recipient` needed — the account's address is filled in automatically.
const { buyUrl } = await transak.buy({
  cryptoAsset: 'ETH',
  fiatCurrency: 'USD',
  fiatAmount: 10_000n // 100 USD (in cents)
})

// Likewise, no `refundAddress` needed for sell.
const { sellUrl } = await transak.sell({
  cryptoAsset: 'ETH',
  fiatCurrency: 'USD',
  cryptoAmount: 100_000_000_000_000_000n // 0.1 ETH (in wei)
})

// An explicit recipient/refundAddress still wins over the account address:
const { buyUrl: toOther } = await transak.buy({
  cryptoAsset: 'ETH',
  fiatCurrency: 'USD',
  fiatAmount: 10_000n,
  recipient: '0xSomeOtherAddress'
})
```

Choosing an account type

The `account` argument accepts three forms:

- **`undefined`** — no wallet is bound. Pass `recipient`/`refundAddress` explicitly on `buy`/`sell`, or omit them to let the Transak widget prompt the user for the address. Ideal for quotes, backend/server usage, or when the destination address comes from elsewhere.
- **`IWalletAccountReadOnly`** — a read-only account. Its `getAddress()` is used to pre-fill the wallet address on `buy`/`sell`.
- **`IWalletAccount`** — a full wallet account. Works the same as a read-only account here, since the buy/sell flow never signs transactions (Transak's widget handles the crypto side).

Only `buy` and `sell` use the account, and only to fill in the wallet address when you don't pass one. `quoteBuy`, `quoteSell`, `getTransactionDetail`, and the `getSupported*` methods don't use it, so `undefined` is fine for those.

💱 Amounts & Units

Following the WDK `FiatProtocol` contract, all amounts cross the interface in **base units** (the smallest unit of the currency/asset), as `number` or `bigint` — for both inputs and quote outputs. This is the same convention as the other WDK fiat modules (e.g. MoonPay), so the same call means the same thing across providers.

- `fiatAmount` — the currency's minor unit (cents). `10_000n` = **$100** (10,000 cents).
- `cryptoAmount` — the asset's on-chain base unit (wei for ETH). `100_000_000_000_000_000n` = **0.1 ETH**; `1_000_000_000_000_000_000n` = 1 ETH.

Prefer `bigint` (the `n` suffix) — crypto base units routinely exceed JavaScript's safe integer range. The module converts these to the standard units Transak's API expects, using each asset's `decimals` and each currency's `roundOff` (read from the supported-currencies lists). Quotes are returned the same way: `cryptoAmount`, `fiatAmount`, and `fee` are `bigint` base units, and `rate` is a decimal string.

🌐 Values & Conventions

Pass `cryptoAsset`, `fiatCurrency`, `network`, and `paymentMethod` **exactly** as Transak spells them — the module matches them case-sensitively and won't fix the casing for you. Get it wrong and you'll see `Cannot find info for cryptoAsset and fiatCurrency`.

```
// ✓ Correct — values match Transak's conventions exactly
await transak.quoteBuy({
  cryptoAsset: 'USDT',   // upper-case symbol
  fiatCurrency: 'USD',   // upper-case ISO 4217 code
  fiatAmount: 10_000n,   // base units (cents) = $100
  config: {
    network: 'ethereum',                // lower-case network name
    paymentMethod: 'credit_debit_card'  // lower-case identifier
  }
})

// ✗ Wrong — the casing isn't fixed up for you, so this throws:
// Cannot find info for cryptoAsset and fiatCurrency
await transak.quoteBuy({
  cryptoAsset: 'usdt',
  fiatCurrency: 'usd',
  fiatAmount: 10_000n,
  config: { network: 'Ethereum', paymentMethod: 'credit_debit_card' }
})
```

Field	Convention	Examples
`cryptoAsset`	Upper-case symbol	`ETH`, `USDT`, `USDC`, `BTC`, `SOL`, `BNB`, `XRP`, `TRX`, `DOGE`, `LTC`
`fiatCurrency`	Upper-case ISO 4217 code	`USD`, `EUR`, `GBP`, `CHF`, `SEK`, `PLN`, `NOK`, `DKK`

`network`	Lower-case network name	`ethereum`, `tron`, `solana`, `bsc`, `polygon`, `mainnet` (BTC/LTC/XRP/DOGE)
`paymentMethod`	Lower-case identifier	`credit_debit_card`, `sepa_bank_transfer`, `gbp_bank_transfer`, `apple_pay`, `google_pay`, `pm_open_banking`, `pm_wire`

> These are just examples. The real list depends on your Transak account and the user's country, so fetch it at runtime (see below) instead of hard-coding values.

Finding the exact values

Call `getSupportedCryptoAssets()` and `getSupportedFiatCurrencies()` to get the exact values. Each crypto asset gives you its `code` (the `cryptoAsset` symbol) and `networkCode` (the `network`):

```
const assets = await transak.getSupportedCryptoAssets()
// [
//   { code: 'ETH',  networkCode: 'ethereum', decimals: 18, name: 'Ethereum', metadata: { … }
//   { code: 'USDT', networkCode: 'ethereum', decimals: 6,  name: 'Tether USD', metadata: { .
//   { code: 'USDT', networkCode: 'tron',     decimals: 6,  name: 'Tether USD', metadata: { .
//   …
// ]

const currencies = await transak.getSupportedFiatCurrencies()
// [
//   { code: 'USD', decimals: 2, name: 'US Dollar', metadata: { … } },
//   { code: 'EUR', decimals: 2, name: 'Euro',      metadata: { … } },
//   …
// ]
```

Payment methods come from each fiat currency's `metadata.paymentOptions` array — use the `id` of the option you want:

```
const [usd] = (await transak.getSupportedFiatCurrencies()).filter(c => c.code === 'USD')
const paymentMethods = usd.metadata.paymentOptions.map(o => o.id)
// e.g. ['credit_debit_card', 'apple_pay', 'google_pay', 'pm_wire', …]
```

Assets on multiple networks

Transak identifies an asset by symbol **and** network, so a symbol like `USDT` can exist on several chains (`ethereum`, `tron`, `solana`, …). For all methods, `network` is optional in `config`; when omitted the module uses the **first** matching entry from the supported-crypto list. Pass `config.network` to pick a specific chain:

```
await transak.buy({
  cryptoAsset: 'USDT',
  fiatCurrency: 'USD',
  fiatAmount: 10_000n, // $100 in cents
  config: { network: 'tron' }
```

```
})
```

📚 API Reference

TransakProtocol

Main class for Transak integration.

Constructor

```
new TransakProtocol(account, config)
```

Parameters:

- `account` (IWalletAccount | IWalletAccountReadOnly | undefined): The wallet account to bind to the protocol. Used only by `buy`/`sell` to auto-fill the wallet address when `recipient`/`refundAddress` is omitted. Pass `undefined` for an unbound instance. See Choosing an account type.
- `config` (object): The protocol config
  - `apiKey` (string): Your Transak partner API key.
  - `widgetUrl` (function, required for `buy`/`sell`): A callback that receives the assembled `widgetParams` object and returns a Transak widget URL. Implement it on your backend by calling Transak's Create Widget URL API (which needs your API secret). `buy` and `sell` throw if it isn't provided; `quoteBuy`, `quoteSell`, and the `getSupported*` methods don't need it.
  - `cacheTime` (number, optional): The duration in milliseconds to cache supported currencies.
  - `environment` ("PRODUCTION" | "STAGING", optional): The environment to use for Transak endpoints and widget URLs. Defaults to "PRODUCTION". Use "PRODUCTION" for live transactions and "STAGING" for testing with non-real funds.

Methods

Method	Description	Returns
`buy(options)`	Generates a widget URL to purchase crypto	`Promise<BuyResult>`
`sell(options)`	Generates a widget URL to sell crypto	`Promise<SellResult>`

`quoteBuy(options)`	Gets a quote for a crypto asset purchase	`Promise<TransakBuyQuote>`
`quoteSell(options)`	Gets a quote for a crypto asset sale	`Promise<TransakSellQuote>`
`getTransactionDetail(txId)`	Retrieves the details of an order	`Promise<TransakTransactionDetail>`
`getSupportedCryptoAssets()`	Retrieves a list of supported crypto assets	`Promise<TransakSupportedCryptoAsset[]>`
`getSupportedFiatCurrencies()`	Retrieves a list of supported fiat currencies	`Promise<TransakSupportedFiatCurrency[]>`
`getSupportedCountries()`	Retrieves a list of supported countries	`Promise<TransakSupportedCountry[]>`

`buy(options)`

Generates a widget URL to purchase crypto.

Options:

- `fiatCurrency` (string): The fiat currency code (e.g., 'USD').
- `cryptoAsset` (string): The crypto asset code (e.g., 'ETH').
- `fiatAmount` (number | bigint, optional): The fiat amount, in base units (e.g. `10_000n` = 100 USD in cents).
- `cryptoAmount` (number | bigint, optional): The crypto amount, in base units (e.g. `1_000_000_000_000_000_000n` = 1 ETH in wei).
- `recipient` (string, optional): The wallet address to receive funds. If not provided, uses the account address.
- `config` (object, optional): Additional Transak widget parameters (including `network`).

Returns `Promise<{ buyUrl: string }>` — the URL your `widgetUrl` callback produced:

```
{
  buyUrl: 'https://global.transak.com/?apiKey=4fcd6904-706b-4aff-bd9d-77422813bbb7&sessionId=
}
```

`sell(options)`

Generates a widget URL to sell crypto.

Options:

- `fiatCurrency` (string): The fiat currency code (e.g., 'USD').
- `cryptoAsset` (string): The crypto asset code (e.g., 'ETH').
- `fiatAmount` (number | bigint, optional): The fiat amount, in base units (e.g. `10_000n` = 100 USD in cents).
- `cryptoAmount` (number | bigint, optional): The crypto amount, in base units (e.g. `1_000_000_000_000_000_000n` = 1 ETH in wei).
- `refundAddress` (string, optional): The wallet address for refunds. If not provided, uses the account address.
- `config` (object, optional): Additional Transak widget parameters (including `network`).

Returns `Promise<{ sellUrl: string }>` — the URL your `widgetUrl` callback produced:

```
{
  sellUrl: 'https://global.transak.com/?apiKey=4fcd6904-706b-4aff-bd9d-77422813bbb7&sessionId
}
```

`quoteBuy(options)`

Gets a quote for a crypto asset purchase.

Options:

- `cryptoAsset` (string): The crypto asset code (e.g. `'ETH'`).
- `fiatCurrency` (string): The fiat currency code (e.g. `'USD'`).
- `fiatAmount` (number | bigint, optional): The fiat amount, in base units (e.g. `10_000n` = 100 USD in cents). Provide this or `cryptoAmount`.
- `cryptoAmount` (number | bigint, optional): The crypto amount, in base units (e.g. `1_000_000_000_000_000_000n`). Provide this or `fiatAmount`.
- `config` (object, optional): Provider-specific extras — `paymentMethod` (e.g. `'credit_debit_card'`) and `network` (resolved from the supported list when omitted).

Returns `Promise<TransakBuyQuote>`. Amounts are `bigint` base units; `rate` is a decimal string; `metadata` is the raw Transak quote:

```
{
  cryptoAmount: 500000000000000000n, // 0.5 ETH, in base units (wei)
  fiatAmount: 100000n,               // 1000.00 USD, in base units (cents)
  fee: 500n,                         // 5.00 USD, in base units (cents)
  rate: '2000',                      // 1 ETH = 2000 USD (standard units)
  metadata: { quoteId: 'q1', conversionPrice: 2000, /* …full Transak quote */ }
}
```

`quoteSell(options)`

Gets a quote for a crypto asset sale.

Options:

- `cryptoAsset` (string): The crypto asset code (e.g. `'ETH'`).

- `fiatCurrency` (string): The fiat currency code (e.g. `'USD'`).
- `cryptoAmount` (number | bigint, **required**): The crypto amount to sell, in base units (e.g. `1_000_000_000_000_000_000n`).
- `config` (object, optional): Provider-specific extras — `paymentMethod` (e.g. `'sepa_bank_transfer'`) and `network` (resolved from the supported list when omitted).

Returns `Promise<TransakSellQuote>` — same shape as `quoteBuy`:

```
{
  cryptoAmount: 500000000000000000n, // 0.5 ETH, in base units (wei)
  fiatAmount: 99500n,                // 995.00 USD, in base units (cents)
  fee: 500n,                         // 5.00 USD, in base units (cents)
  rate: '2000',                      // 1 ETH = 2000 USD (standard units)
  metadata: { quoteId: 'q2', /* …full Transak quote */ }
}
```

`getTransactionDetail(txId)`

Retrieves the details of an order.

Parameters:

- `txId` (string): The Transak order ID.

Returns `Promise<TransakTransactionDetail>`. `status` is normalised to `'in_progress' | 'failed' | 'completed'`; `metadata` is the raw Transak order:

```
{
  status: 'completed',
  cryptoAsset: 'ETH',
  fiatCurrency: 'USD',
  metadata: { id: 'o1', status: 'COMPLETED', /* …full Transak order */ }
}
```

`getSupportedCryptoAssets()`

Retrieves a list of supported crypto assets. `code` is the `cryptoAsset` symbol and `networkCode` is the `network` — use both exactly as returned (see Values & Conventions). The full Transak object is under `metadata`.

Returns `Promise<Array<{ code, networkCode, decimals, name, metadata }>>`:

```
[
  { code: 'ETH', networkCode: 'ethereum', decimals: 18, name: 'Ethereum', metadata: { /* … */
  { code: 'USDT', networkCode: 'tron', decimals: 6, name: 'Tether USD', metadata: { /* … */ }
]
```

`getSupportedFiatCurrencies()`

Retrieves a list of supported fiat currencies. `code` is the `fiatCurrency` code. Available `paymentMethod` identifiers are found in `metadata.paymentOptions[].id`.

Returns `Promise<Array<{ code, decimals, name, metadata }>>`:

```
[
  { code: 'USD', decimals: 2, name: 'US Dollar', metadata: { /* …incl. paymentOptions */ } },
  { code: 'EUR', decimals: 2, name: 'Euro', metadata: { /* … */ } }
]
```

`getSupportedCountries()`

Retrieves a list of supported countries.

Returns `Promise<Array<{ code, isBuyAllowed, isSellAllowed, name, metadata }>>`:

```
[
  { code: 'US', isBuyAllowed: true, isSellAllowed: true, name: 'United States', metadata: {
  { code: 'GB', isBuyAllowed: true, isSellAllowed: true, name: 'United Kingdom', metadata: {
]
```

📝 Notes

- Works with the networks and currencies your Transak account supports.
- This package covers the basics. For the full list of widget parameters, see the Transak query parameters docs.
- Get your `apiKey` from the Transak Partner dashboard.
- Test the full buy/sell flow in the `STAGING` environment before going live.

🔗 Creating the widget URL (backend)

`buy` and `sell` build a `widgetParams` object and hand it to your `widgetUrl` callback. That callback runs on your backend, where your API secret is safe, and turns the params into a widget URL using Transak's API-based flow (passing params directly in the URL is deprecated). It's two calls:

```
// On your backend — never ship the API secret to the client.
async function createWidgetUrl (widgetParams) {
  // 1. Get a partner access token (cache it until it expires).
  const tokenRes = await fetch('https://api.transak.com/partners/api/v2/refresh-token', {
    method: 'POST',
    headers: { 'api-secret': process.env.TRANSAK_API_SECRET, 'content-type': 'application/jso
    body: JSON.stringify({ apiKey: process.env.TRANSAK_API_KEY })
  })
  const { data: { accessToken } } = await tokenRes.json()

  // 2. Create the widget session and return its URL.
  const sessionRes = await fetch('https://api-gateway.transak.com/api/v2/auth/session', {
```

```
    method: 'POST',
    headers: { 'access-token': accessToken, 'content-type': 'application/json' },
    body: JSON.stringify({ widgetParams })
  })
  const { data: { widgetUrl } } = await sessionRes.json()
  return widgetUrl // valid for 5 minutes, single use
}
```

Notes:

- Keep the API secret on the backend, never in client code.
- The returned widget URL is valid for **5 minutes** and each session can be used **once** — create a fresh one per flow.
- For staging, use `https://api-stg.transak.com` and `https://api-gateway-stg.transak.com`.

🛠 Development

Building

```
# Install dependencies
npm install

# Build TypeScript definitions
npm run build:types

# Lint code
npm run lint

# Fix linting issues
npm run lint:fix
```

Testing

```
# Run tests
npm test

# Run tests with coverage
npm run test:coverage
```

📄 License

This project is licensed under the Apache License 2.0 - see the LICENSE file for details.

Contributing

Contributions are welcome! Please feel free to submit a Pull Request.

🆘 Support

For support, please open an issue on the GitHub repository.